

Bounds for the String Editing Problem

C. K. WONG AND ASHOK K. CHANDRA

IBM Thomas J. Watson Research Center, Yorktown Heights, New York

ABSTRACT. The string editing problem is to determine the distance between two strings as measured by the minimal cost sequence of deletions, insertions, and changes of symbols needed to transform one string into the other. The longest common subsequence problem can be viewed as a special case. Wagner and Fischer proposed an algorithm that runs in time $O(nm)$, where n, m are the lengths of the two strings. In the present paper, it is shown that if the operations on symbols of the strings are restricted to tests of equality, then $O(nm)$ operations are necessary (and sufficient) to compute the distance.

KEY WORDS AND PHRASES. string editing, longest common subsequence, bounds for computing edit distance

CR CATEGORIES. 3.79, 4.12, 4.22, 5.23, 5.25, 5.39

1. The Problem

Given a string $a = a_1, a_2, \dots, a_n$ from an alphabet Σ , we consider the following "editing operations" on a :

- (i) deleting a symbol from any position, say i ($1 \leq i \leq n$), to give $a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n$;
- (ii) inserting a symbol, say b ($b \in \Sigma$) at position i ($0 \leq i \leq n$), to give $a_1, \dots, a_i, b, a_{i+1}, \dots, a_n$;
- (iii) changing a symbol, say the i th symbol ($1 \leq i \leq n$) to b ($b \in \Sigma$), to give $a_1, \dots, a_{i-1}, b, a_{i+1}, \dots, a_n$.

With fixed positive costs C_D, C_I, C_C associated with deleting, inserting, and changing any symbol, and strings $a = a_1, \dots, a_n$ and $b = b_1, \dots, b_m$ given, the problem is to determine a sequence of editing operations which, when applied to a , produces b so that the sum of the individual costs is minimized.

Without loss of generality it may be assumed that $C_C \leq C_D + C_I$ and that $n \geq m \geq 1$.

It may be observed that, when $C_C = C_D + C_I$, the subsequence of (not necessarily adjacent) symbols of a that remain unchanged during a minimal cost edit transforming a into b is precisely the longest common subsequence of a and b . Thus our lower and upper bounds also apply to the longest common subsequence problem.

The editing problem was considered by Wagner and Fischer in [1], where they demonstrated an $O(nm)$ algorithm to solve the problem. We show that, under a restrictive model of computation where the only questions that can be asked about the strings are whether two symbols are equal or unequal, $O(nm)$ is also a lower bound. While this does not give a definitive lower bound for a more general class of algorithms, it does say that certain techniques will not work for decreasing the upper bound.

Copyright © 1976, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

Authors' address. Computer Sciences Department, IBM Thomas J. Watson Research Center, P.O. Box 218, Yorktown Heights, NY 10598.

2. The Model

With positive costs $C_D, C_I, C_C, C_C \leq C_D + C_I$, and strings $a = a_1, \dots, a_n, b = b_1, \dots, b_m, n \geq m$ given, the minimal editing cost can be determined as follows [1]. Let d be a function $d : \{0, 1, \dots, n\} \times \{0, 1, \dots, m\} \rightarrow R$ such that

$$d(0, j) = jC_I, \quad (1a)$$

$$d(i, 0) = iC_D, \text{ and} \quad (1b)$$

$$d(i+1, j+1) = \min\{d(i, j+1) + C_D, d(i+1, j) + C_I, d(i, j) + (\text{if } a_{i+1} = b_{j+1} \text{ then } 0 \text{ else } C_C)\}. \quad (1c)$$

$d(i, j)$ is the minimal editing cost to change sequence $a_1, \dots, a_i$ into $b_1, \dots, b_j$, and $d(n, m)$ is the desired editing cost.

We assume that the alphabet Σ is arbitrarily large ($|\Sigma| \geq n + m$), and that the only questions that can be asked about the input sequences a, b are " $a_i = b_j$ ", " $a_i = a_j$ ", " $b_i = b_j$ ". Let $M_{C_D, C_I, C_C}(n, m)$ be the minimal number of questions that must be asked in the worst case to determine a best editing sequence for all strings a, b of lengths n, m respectively ($n \geq m$). We will frequently omit subscripts, writing $M(n, m)$ for convenience.

Notation. We write v for $C_C/(C_D + C_I)$. Obviously $0 < v \leq 1$.

3. A Lower Bound

THEOREM 1.

$$M(n, m) \geq m(n - m + 1) + 2 \sum_{\substack{1 \leq k \leq m-1 \\ m - (k/v) \geq 0}} \lceil m - k/v \rceil. \quad (2)$$

$$\text{COROLLARY 1.}^1 M(n, m) \geq m(n - m) + vm^2 - ((1/v) - 1). \quad (3)$$

$$\text{COROLLARY 2. If } v = 1 \text{ (common subsequence problem), } M(n, m) \geq mn. \quad (4)$$

$$\text{If } C_D = C_I = C_C, M(n, m) \geq mn - m^2/2. \quad (5)$$

Equations (3)–(5) follow from (2) above.

PROOF OF THEOREM 1. Consider the problem of computing $d(n, m)$ when we know in advance that whenever $i \neq j, a_i \neq a_j$, and whenever $i \neq j, b_i \neq b_j$. Consider the situation when all tests " $a_i = b_j$ " turn out to be false, and let T be the set of tests made. We show that the answer could be incorrect unless $|T|$ is greater than or equal to the right-hand side of (2).

First, the output must be

$$C_0 = mC_C + (n - m)C_D, \quad (6)$$

which is the correct output if $a_i \neq b_j$ for all i, j (a case consistent with the answers of T).

Second, let

$$T_0 = \{ "a_i = b_j" : i \geq j \geq 0, n - i \geq m - j \geq 0 \}. \quad (7)$$

Clearly $|T_0| = m(n - m + 1)$. $T_0 \subset T$, because if not, some test " $a_{i_0} = b_{j_0}$ " $\in T_0$ is not made in the algorithm. If the symbols a_{i_0} and b_{j_0} are the same, then a can be transformed into b using the following editing operations, with cost only $(m - 1)C_C + (n - m)C_D$ (let $\beta = i_0 - j_0$):

- (i) delete a_i , for $(1 \leq i \leq \beta) \vee (\beta + m + 1 \leq i \leq n)$;
- (ii) change a_i to $b_{i-\beta}$, for $(\beta + 1 \leq i \leq \beta + m) \wedge (i \neq i_0)$;

and hence a contradiction.

Third, for $1 \leq k \leq m - 1$, let

$$U_k = \{ "a_i = b_{i+k}" : 1 \leq i < i + k \leq m \}. \quad (8)$$

¹ R. A. Wagner pointed out a slightly improved version of this corollary: $M(n, m) \geq m(n - m) + vm^2$.

We claim that $|T \cap U_k| \geq \lfloor m - k/v \rfloor$. Let $u_k = |T \cap U_k|$, and suppose all tests in $U_k - T$ would have yielded true. Consider the following editing operations transforming a into b :

- (i) insert b_j , for $1 \leq j \leq k$,
- (ii) change a_i to b_{i+k} , for " $a_i = b_{i+k}$ " $\in T \cap U_k$,
- (iii) delete a_i , for $m - k + 1 \leq i \leq n$.

The cost is $C = kC_I + u_k C_C + (n - m + k)C_D$. Since the output is C_0 (see (6)), it must be the minimal cost, i.e. $C \geq C_0$. It follows that $u_k \geq m - k/v$.

Fourth, for $1 \leq k \leq m - 1$, let

$$V_k = \{ "a_{n-m+k+i} = b_i" : 1 \leq i < i+k \leq m \}. \quad (9)$$

As above, we can show that $|T \cap V_k| \geq \lfloor m - k/v \rfloor$. Since all sets T_0, U_k, V_k are disjoint, the theorem is proved. $\square$

4. An Upper Bound

THEOREM 2. $M(n, m) \leq mn - \lfloor m(1 - v) \rfloor \cdot \lfloor m(1 - v) + 1 \rfloor$. (10)

COROLLARY 3. $M(n, m) \leq m(n - m) + v(2 - v)m^2 + m$. (11)

COROLLARY 4. If $v = 1$, $M(n, m) \leq mn$. (12)

If $C_D = C_I = C_C$, $M(n, m) \leq mn - (m^2 - 1)/4$. (13)

Equations (11)–(13) follow from (10). Since $v > 0$, the upper bound (11) is within a factor of 2 of the lower bound (3) in the limit as $m \rightarrow \infty$.

PROOF OF THEOREM 2. Below, we use the notations in (6)–(9). Since $|U_k| = |V_k| = m - k$, the set

$$T = T_0 \cup U_1 \cup V_1 \cup \dots \cup U_{k_0} \cup V_{k_0}, \quad (14)$$

where $k_0 = \lfloor mv \rfloor - 1$, has $|T|$ equal to the right-hand side of (10), and we claim that T is adequate to compute the minimal cost editing sequence.

The minimal possible cost of transforming $a = a_1, \dots, a_p$ into $b = b_1, \dots, b_q$ (we do not insist that $p \geq q$) is

$$C_{\min}(p, q) = \begin{cases} (p - q)C_D & \text{if } p \geq q, \\ (q - p)C_I & \text{if } p < q, \end{cases} \quad (15)$$

and the maximal possible cost is

$$C_{\max}(p, q) = C_C \min(p, q) + C_{\min}(p, q). \quad (16)$$

Any editing sequence that transforms a into b and leaves a_i unchanged such that a_i becomes b_j (of course $a_i = b_j$) has cost at least $C = C_{\min}(i - 1, j - 1) + C_{\min}(n - i, m - j)$. If " $a_i = b_j$ " $\in U_k \cup V_k$, then $C = kC_I + (n - m + k)C_D$, and if $k \geq mv$, then $C \geq C_{\max}(n, m)$; i.e. there exists an optimal editing sequence that transforms no such a_i into such a b_j .

Then, as in the construction of [1], we can compute a minimal cost editing sequence using only T : compute a function d' like the recurrence (1c) for d , except $d'(i, j)$ is initialized to ∞ (and not recomputed) for all " $a_i = b_j$ " $\in U_k \cup V_k, k > k_0$. $\square$

5. Concluding Remarks

We have demonstrated that if one is restricted to tests of equality only, a lower bound $O(n^2)$ is obtainable for the string editing problem (assume strings are of equal length n). Wagner and Fischer's algorithm [1] achieves $O(n^2)$ (see also [2, 3]). Our lower and upper bounds apply to the special case of the string editing problem, namely, the longest common subsequence problem.

The lower bound does not apply to a more general class of algorithms. For example, it is usually possible to well-order the alphabet Σ (e.g. by a lexicographic ordering on

the representation of the symbols). If in our model one could also ask questions like " $a_i \leq b_j$," then $O(n \log n)$ such tests would suffice. Also, if $|\Sigma|$ is constant (and "small enough") and one could test for equality with elements in Σ , then $O(n)$ such tests would be sufficient to solve the string editing problem.

Also, our lower bound is not improved when one considers the problem of finding the longest subsequence common to k given strings each of length n (k fixed). Here the lower bound of $O(n^2)$ may be compared with the obvious $O(n^k)$ algorithm for solving the problem.

Recently we learned that Aho, Hirschberg, and Ullman [4] studied another aspect of this problem. They obtained lower and upper bounds for the longest common subsequence problem in terms of the size of the alphabet in a model much like ours.

REFERENCES

1. WAGNER, R. A., AND FISCHER, M. J. The string-to-string correction problem. *J. ACM* 21, 1 (Jan 1974), 168-173.
2. SELLERS, P. H. An algorithm for the distance between two finite sequences. *J. Combin. Theory (A)*, 16 (1974), 253-258.
3. SANKOFF, D. Matching sequences under deletion insertion constraints. *Proc. Nat. Acad. Sci. USA* 69 (1972), 4-6.
4. AHO, A. V., HIRSCHBERG, D. S., AND ULLMAN, J. D. Bounds on the complexity of the maximal common subsequence problem. *Proc. 15th Ann. IEEE Symp. on Switching and Automata Theory*, 1974, pp. 104-109.

RECEIVED JUNE 1974; REVISED MAY 1975